

PRÁCTICO 2

INF329 SA-COMPILADORES. GESTIÓN 1-2022

1. Escribir una BNF S que reconozca TODAS las cadenas gramaticales formadas solamente con los **tokens** **a** y **b** y que cumple la siguiente condición:

En ningún lugar de la cadena se tienen 2 o más tokens, “b” consecutivos.

Ejemplos:

```
a           //Bien
b           //Bien
abaabaaa   //Bien
aaaa       //Bien
ababbba    //Mal. Hay dos b's consecutivas.
```

2. Escribir una BNF “Expr” la cual especifique una expresión aritmética sencilla de NUM’s, con solamente los operadores **+** y **-**.

Ejemplos:

```
235           //Un solo NUM es una Expr
23 + 16
23 - 15
16 + 15 - 12 - 20 + 1
```

3. Escribir una BNF “Expr” la cual especifique una expresión aritmética sencilla de NUM’s, con solamente los operadores **+** y **-**, que cumple la siguiente condición:

Si la Expr tiene 2 ó más NUM’s, los operadores deben alternarse estrictamente.

Ejemplos:

```
235           //Un solo NUM es una Expr
23 + 16
23 - 15
16 + 15 - 12   //Se alterna +, -
11 - 15 + 12 - 4 + 60 //Se alterna -, +, -, +
```

4. Escriba una BNF “Lista” que especifique una lista de PROLOG, cuyos elementos son ID’s y NUM’s, separados por comas.

Ejemplos:

```
[ ]           //Lista Vacía
[23]          //Lista de un elemento
[Z]           //Lista de un elemento
[23, w, y, 15, 20] //Lista de 5 elementos.
```

5. Especificar en una BNF “Lista”, la cual modele una colección de NUM’s, separados con comas y encerrada entre corchetes. Tome en cuenta que la Lista puede contener sub-listas (lista dentro de otra lista). Ejemplos:

```
[ ]           //Lista Vacía
[23]          //Lista de un solo NUM
[18, 15, 60]  //Lista de 3 NUM's.
[8, [20, 5], 60, [ ], 34 ] //Lista de 5 elementos (tiene 2 sub-listas)
[8, [35, 5, [6], 12, [ ], 3], 37, [20 ], 17] //Note que las sub-Listas también
                                              //pueden contener sub-listas
```

No es necesario que su BNF esté en LL(1).

6. (BNF) Un lenguaje usa **solamente** los tokens **0** y **1**. Escriba una BNF “Binario”, la cual reconozca **todas** las cadenas binarias que tengan **longitud impar**.

No es necesario que su BNF esté en LL(1).

7. Calcule los conjuntos First de:

```
A → A C / B | B C ID | λ
B → C | ( B )
C → NUM + C | ID - C | λ
```

Muestre los cálculos realizados

8. Calcule los conjuntos First de:

```
S → Q S P NUM | P S ( ID ) | Q P
P → Q P * | λ
Q → + ID | λ
```

Muestre los cálculos realizados

9. Dada la BNF “Expr”:

```
Expr → Expr + Termino | Expr - Termino | Termino
Termino → Termino * Factor | Termino / Factor | Factor
Factor → NUM | ID | (Expr) | - Factor
```

(a) Dibuje un Árbol de Análisis Sintáctico (AAS), para la frase:

$45*(23+5*4) + 2*4*3 + 5$

(b) Escriba un Parser para esta BNF

10. La siguiente BNF “Expr”, especifica correctamente una suma de $n \geq 1$, NUM’s

$\text{Expr} \rightarrow \text{NUM} + \text{Expr} \mid \text{NUM} - \text{Expr} \mid \text{NUM}$

Escriba un Parser que, al margen de chequear la sintaxis de la Expr, también calcule el valor de la misma. Por ejemplo:

$12 - 4 + 8$ $\text{Expr}() = 0$
 $40 + 20 - 10 - 50 + 6$ $\text{Expr}() = 6$

11. La siguiente BNF “Expr”, especifica correctamente una suma de productos de $n \geq 1$ términos (sumandos), los cuales tienen la forma NUM*NUM o -NUM*NUM

$\text{Expr} \rightarrow \text{Termino Op Expr} \mid \text{Termino}$
 $\text{Termino} \rightarrow \text{NUM} * \text{NUM} \mid - \text{NUM} * \text{NUM}$
 $\text{Op} \rightarrow + \mid -$

Escriba un Parser, que al margen de chequear la sintaxis, calcule el valor de la expresión.

Por ejemplo, si el programa fuente es

$-23 * 1 + 12 * 2 - -10 * 20$

Entonces $\text{Expr}() = 201$

Note que cuando se dice “suma”, se refiere a las operaciones “+” y “-”

12. La siguiente BNF “Expr”, especifica correctamente una expresión aritmética que usa sumas (SUM) y multiplicaciones (MUL) anotada en un estilo funcional.

$\text{Expr} \rightarrow \text{SUM}(\text{Elem}, \text{Elem MasElem}) \mid \text{MUL}(\text{Elem}, \text{Elem MasElem})$
 $\text{Elem} \rightarrow \text{NUM} \mid \text{Expr}$
 $\text{MasElem} \rightarrow , \text{Elem MasElem} \mid \lambda$

Escriba un **Parser**, que al margen de chequear la sintaxis de la Expr, encuentre su *valor*.

Por ejemplo:

$\text{SUM}(13, 5, 10) = 13 + 5 + 10 = 28$
 $\text{SUM}(20, 6, \text{MUL}(60, 5, 1), 25) = 20 + 6 + 60 * 5 * 1 + 25 = 351$

13. La siguiente BNF “EBoole” especifica una expresión booleana sencilla de ID’s (o negación de ID’s) con el operador OR.

$\text{EBoole} \rightarrow \text{Literal OR Eboole} \mid \text{Literal}$
 $\text{Literal} \rightarrow \text{ID} \mid \text{NOT ID}$

Escriba un Parser, que al margen de chequear la sintaxis, calcule el valor de la expresión booleana.

Por ejemplo: Sabiendo que $b = \text{false}$, $c = \text{false}$, $d = \text{true}$ y que el programa fuente es:

$b \text{ or not } c \text{ or } d$

entonces $\text{EBoole}() = \text{true}$.

Para obtener el valor de un **ID**, use la función booleana `getValor()` de la class `Token`. Por ejemplo, si un token `T` es un `ID`, o sea `T=ID`, podemos obtener el valor de éste `ID` (`true` o `false`), llamando a `T.getValor()`.